DAY 4

Python + Generative AI

Theme: Build Your Own AI Assistant

Student Study Guide | Beginner to Intermediate

What you will learn today

Part	Topics	Outcome
1. Generative AI Fundamentals	What is Generative AI, LLMs, tokens, context, prompts, model parameters, API-based AI	You can explain how an AI chatbot works
2. Prompt Engineering	Role prompting, zero-shot and few-shot, structured prompts, context injection, output formatting, templates	You can write prompts that give reliable answers
3. Python + AI APIs	API keys, environment variables, requests, responses, error handling, conversation history	You can call an AI model from Python safely
4. AI Applications	Chatbots, resume analysis, content generation, study assistants, interview preparation	You know what you can build
5. Hands-on Project	Complete Python AI Assistant with history and error handling	A working project for your portfolio

NOTE: Prerequisite

Basic Python: variables, loops, functions, lists and dictionaries. No earlier AI knowledge is needed.

Table of Contents

How to Use This Guide 3

What you need before starting . . . 3

Today's learning path . . . 3

Part 1: Generative AI Fundamentals 4

1.1 What is Generative AI? . . . 4

1.2 Types and Examples of Generative AI . . . 4

Applications of Generative AI . . . 5

1.3 Large Language Models (LLMs) . . . 5

1.4 Tokens . . . 6

1.5 Context and the Context Window . . . 7

1.6 Prompts . . . 8

1.7 Model Parameters . . . 9

1.8 API-Based AI . . . 10

Part 2: Prompt Engineering 11

2.1 What is Prompt Engineering? . . . 11

2.2 Role Prompting . . . 11

2.3 Zero-Shot and Few-Shot Prompting . . . 12

2.4 Structured Prompting . . . 14

2.5 Context Injection . . . 14

2.6 Output Formatting . . . 15

2.7 Prompt Templates . . . 16

2.8 Prompt Engineering Cheat Sheet . . . 16

Part 3: Python + AI APIs 17

3.1 How an API Request Works . . . 17

3.2 API Keys . . . 17

3.3 Environment Variables . . . 18

3.4 Sending Your First Request . . . 19

3.5 Receiving and Reading Responses . . . 20

3.6 Error Handling . . . 21

3.7 Conversation History . . . 22

Part 4: AI Applications 24

4.1 AI Chatbots . . . 24

4.2 Resume Analysis . . . 24

4.3 Content Generation . . . 26

4.4 Study Assistants . . . 27

4.5 Interview Preparation . . . 27

Part 5: Hands-On Project - Python AI Assistant 29

5.1 Project Goal . . . 29

5.2 The Complete Code . . . 29

5.3 Running the Program . . . 33

5.4 Troubleshooting 34
5.5 Challenges to Extend the Project 34
5.6 Quick Revision Questions 34
5.7 Glossary 35
5.8 Day 4 Summary 35

How to Use This Guide

This guide is written for students who already know basic Python (variables, loops, functions, lists, dictionaries). Every topic follows the same pattern so that it is easy to revise:

1. **Definition** - what the idea means in simple words.
2. **Explanation** - why it matters and how it works.
3. **Diagram** - a picture to remember the idea.
4. **Code + Output** - small programs you can type and run.
5. **Applications** - where the idea is used in real life.

> **NOTE: Real vs sample output**
> Output boxes marked **real output** were produced by actually running the code while this PDF was built. Output boxes marked **sample output** come from a live AI model, so your own result will use different words every time.

What you need before starting

Requirement	Details
Python 3.9 or newer	Check with `python --version` in your terminal
Code editor	VS Code or PyCharm
Internet connection	Needed only when your program calls the AI API
API key	Created from your AI provider's developer console (free credits or a small paid balance may be needed)
Libraries	`pip install anthropic python-dotenv`

Today's learning path



Figure 1: The five steps of Day 4

> **TIP: Study tip**
> Do not just read the code. Type it yourself, change one thing, and run it again. Changing prompts and parameters and watching the effect is the fastest way to learn AI programming.

Part 1: Generative AI Fundamentals

1.1 What is Generative AI?

DEFINITION: Generative AI
Generative AI is a type of artificial intelligence that can **create new content** - such as text, images, music, video, or computer code - by learning patterns from huge amounts of existing data.

Older AI systems mostly *predicted* or *classified*: "Is this email spam?" or "What will the house price be?". Generative AI *produces*: "Write an email to my teacher" or "Draw a cat wearing sunglasses". The output is not copied from a database; it is newly generated, one small piece at a time.

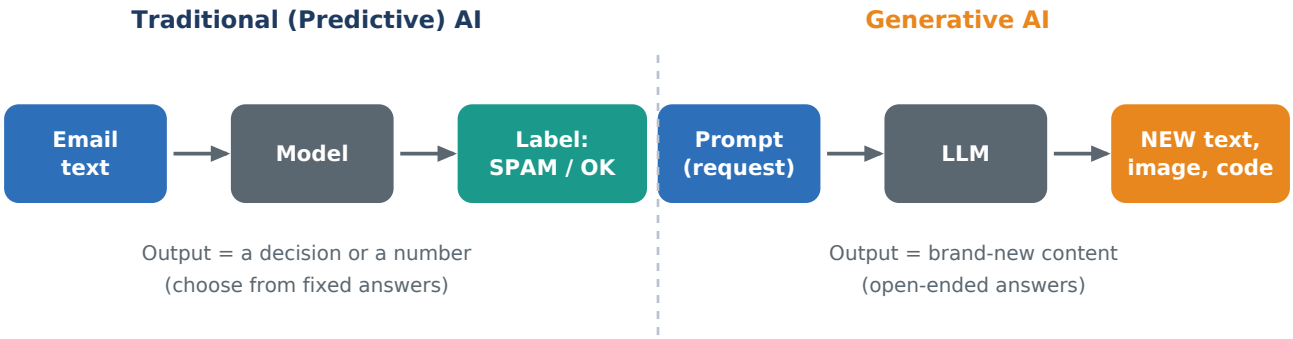


Figure 2: Predictive AI chooses from fixed answers; Generative AI creates new content

Where does Generative AI sit inside AI?

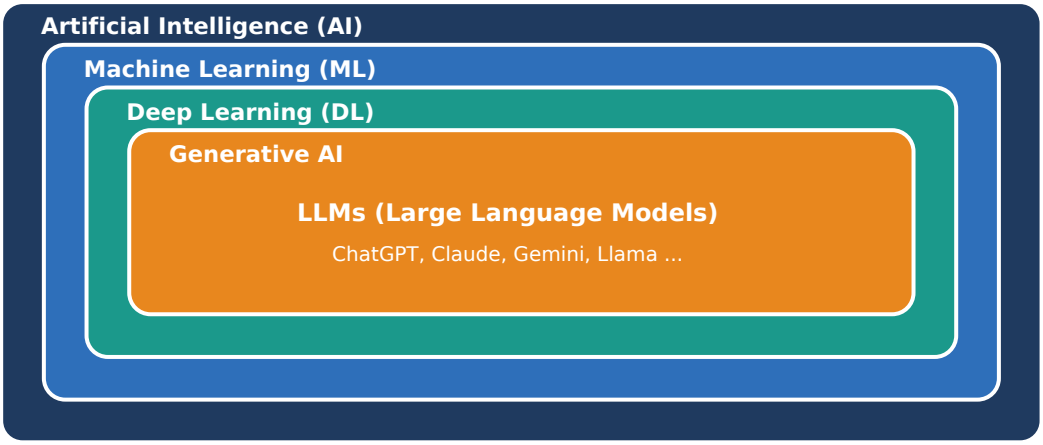


Figure 3: Generative AI is a part of Deep Learning, which is a part of Machine Learning, which is a part of AI

1.2 Types and Examples of Generative AI

Type	What it creates	Example input (prompt)	Example tools
Text	Essays, emails, answers, summaries	"Summarise this article in 3 lines"	Claude, ChatGPT, Gemini
Code	Python, SQL, HTML programs	"Write a function to reverse a string"	Claude Code, GitHub Copilot
Image	Pictures, logos, posters	"A robot teaching in a classroom"	DALL-E, Midjourney, Stable Diffusion
Audio	Speech, music, voice clones	"Read this text in a calm voice"	ElevenLabs, Suno
Video	Short video clips	"Waves hitting a rocky beach"	Sora, Veo, Runway

Example: one idea, three outputs

Prompt	Generated result (shortened)
"Explain photosynthesis to a 10-year-old."	Plants are like tiny kitchens. They take sunlight, water and air and cook their own food...
"Write a Python function that adds two numbers."	`def add(a, b): return a + b`
"Write a 2-line slogan for a tea shop."	"Sip slowly, smile often. Every cup is a little holiday."

Applications of Generative AI

- **Education:** personal tutors, quiz makers, doubt solving.
- **Software:** code completion, bug explanation, test writing.
- **Business:** customer-support chatbots, report summaries, marketing copy.
- **Healthcare:** summarising long medical notes for doctors (always reviewed by humans).
- **Careers:** resume feedback, mock interviews, cover letters.

WARNING: Limitations to remember

Generative AI can sound confident and still be **wrong** (this is called a *hallucination*). It can also repeat bias present in its training data. Always check important facts, and never paste passwords or private data into a chat.

1.3 Large Language Models (LLMs)

DEFINITION: LLM

A **Large Language Model** is a very big neural network trained on enormous amounts of text. Its core skill is simple: **predict the next piece of text**. Because it has seen so much language, this simple skill produces answers, stories and code.

"Large" refers to two things: the size of the training data (books, websites, code) and the number of internal numbers called **parameters** (billions of them) that the model adjusts while learning. Popular LLM families include Claude, GPT, Gemini and Llama.

How an LLM answers your question

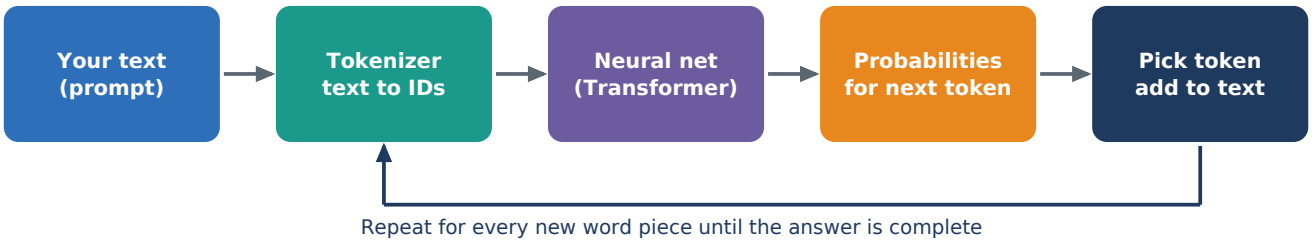


Figure 4: The LLM repeats one loop - predict the next token, add it, predict again

1. Your prompt is split into **tokens** (small pieces of text) and converted to numbers.
2. The neural network reads all the numbers and calculates a **probability for every possible next token**.
3. One token is chosen (usually a likely one) and added to the text.
4. The loop repeats until the model decides the answer is finished.

Example: next-token prediction

Suppose the prompt is **"The capital of France is"**. The model gives every possible next token a probability:



Figure 5: 'Paris' has the highest probability, so it is almost always chosen

The model is not "looking up" the answer in a table. It learned from millions of sentences that "Paris" almost always follows this phrase. That is why the same ability also lets it write poems: it keeps choosing sensible next tokens.

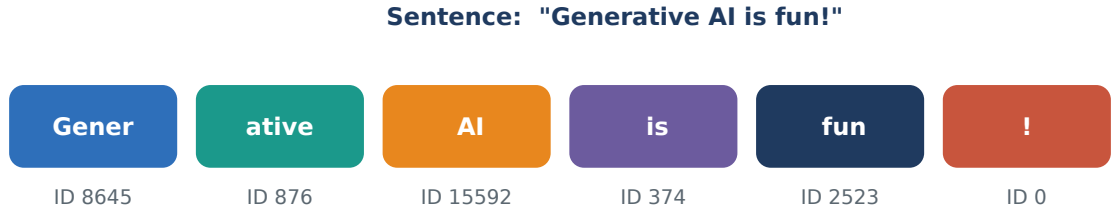
Strengths and weaknesses of LLMs

Strengths	Weaknesses
Understands and writes natural language	May hallucinate (invent) facts
Works on many tasks without retraining	Knowledge stops at its training date
Can follow instructions and formats	Cannot remember past chats unless you send them again
Available through a simple API	Costs money per use and needs internet

1.4 Tokens

DEFINITION: Token

A **token** is the small unit of text an LLM reads and writes. It may be a whole word, part of a word, a space plus a word, or a punctuation mark. As a rule of thumb, **1 token is about 4 characters or 3/4 of an English word**.



The model never sees letters - only these numbers (IDs). IDs shown here are illustrative.

Figure 6: A sentence is cut into tokens and each token becomes a number

Why do tokens matter?

- **Cost:** AI providers charge per token (both input tokens and output tokens).
- **Limits:** every model has a maximum number of tokens it can handle at once (the context window).
- **Speed:** more tokens means a slower response.

▶ **Python code**

```
text = "Generative AI is fun!"
words = text.split()
print("Words                :", words)
print("Number of words      :", len(words))
print("Number of characters :", len(text))
print("Rough token estimate :", round(len(text) / 4))
```

▶ **Output (real output from running this code)**

```
Words                : ['Generative', 'AI', 'is', 'fun!']
Number of words      : 4
Number of characters : 21
Rough token estimate : 5
```

NOTE: Estimate only

The characters divided by 4 trick is only an approximation. The exact count depends on the model's tokenizer. After each API call, the response tells you the real number of tokens used.

Cost estimation example

Providers publish a price per **million tokens**. The prices below are **made-up numbers for practice** - always check your provider's pricing page.

▶ **Python code**

```
def estimate_tokens(text):
    return max(1, len(text) // 4)

prompt = "Explain recursion in simple words with one example."
expected_reply_tokens = 200

price_in = 3.00     # example: dollars per 1 million input tokens
price_out = 15.00   # example: dollars per 1 million output tokens

in_tokens = estimate_tokens(prompt)
cost = (in_tokens * price_in + expected_reply_tokens * price_out) / 1_000_000

print("Input tokens :", in_tokens)
print("Output tokens:", expected_reply_tokens)
print(f"Estimated cost: ${cost:.6f}")
```

▶ **Output (real output from running this code)**

```
Input tokens : 12
Output tokens: 200
Estimated cost: $0.003036
```

1.5 Context and the Context Window

DEFINITION: Context

Context is all the text the model can see when it writes a reply: the system instructions, the earlier conversation, your new question, and any documents you include. The maximum amount it can handle at once is the **context window**, measured in tokens.

Context window = the model's working memory (example size: 8,000 tokens)



If everything together is bigger than the limit, old messages must be cut or summarised.

Figure 7: Everything shares one fixed-size window

Important facts about context

- The model has **no memory between API calls**. If you want it to remember, you must send the earlier messages again.
- A bigger context window lets you include long documents (for example a whole resume plus a job description).
- If the conversation grows too big, you must **trim** old messages or **summarise** them. You will do this in the project.
- The model pays attention best to instructions placed clearly at the start or end of the prompt.

Example

Call	What you send	What the AI can answer
1	"My name is Ravi."	"Nice to meet you, Ravi!"
2 (no history sent)	"What is my name?"	"I don't know your name."
2 (history sent)	Call 1 messages + "What is my name?"	"Your name is Ravi."

1.6 Prompts

DEFINITION: Prompt
A **prompt** is the input you give to the AI: a question, an instruction, examples, or a mix of all three. The quality of the answer depends strongly on the quality of the prompt.

Chat-style AI APIs organise a prompt into **messages**, each with a **role**:

Role	Who writes it	Purpose
system	You, the developer	Sets the AI's personality, rules and task. Sent with every request.
user	The person chatting	The question or instruction.
assistant	The AI (or you, to show examples)	The AI's earlier replies.

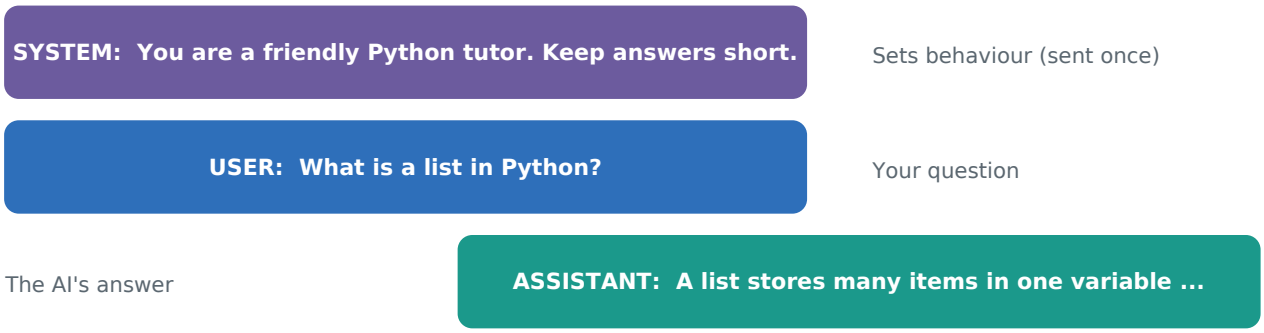


Figure 8: A conversation is a list of role-tagged messages

EXAMPLE: Bad vs good prompt
Bad: "Tell me about Python." (too vague - history? snake? language?)
Good: "Explain the Python programming language to a first-year student in 5 bullet points, each under 15 words."

1.7 Model Parameters

When you call an AI API you can adjust settings called **parameters**. They do not change what the model knows; they change *how* it responds.

Parameter	Meaning	Typical values	Use it when
model	Which AI model to use	Name from the docs	Bigger = smarter but costlier; smaller = faster and cheaper
max_tokens	Maximum length of the reply	100 - 2000	Limit cost and keep answers short
temperature	How random the word choice is	0.0 - 1.0	Low for facts and code; high for stories and ideas
top_p	Choose only from the most likely words that add up to p	0.1 - 1.0	Another way to control randomness (change one of temperature or top_p, not both)
system	Standing instructions	Any text	Define role, tone and rules
stop sequences	Text that makes the model stop	e.g. "END"	Cut output at a marker

Temperature explained with a picture

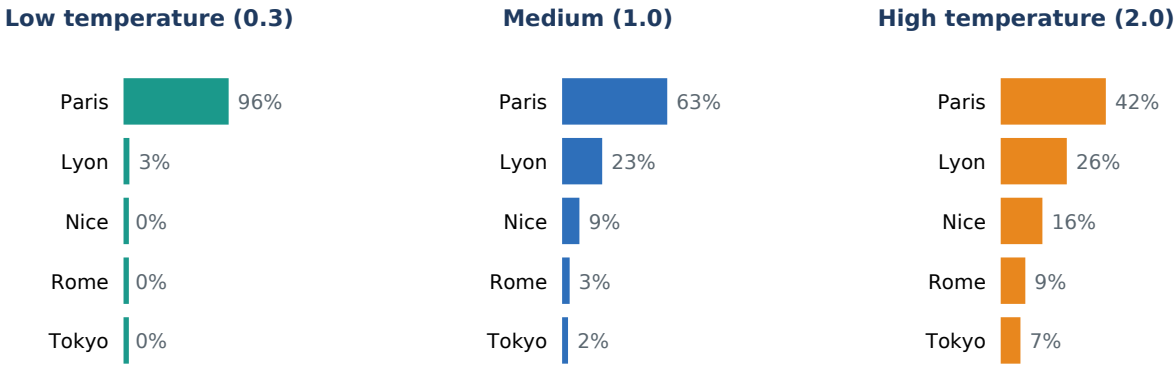


Figure 9: Low temperature makes the top word dominant; high temperature spreads the probability

Temperature demonstration (pure Python)

This small program shows what temperature does to the probabilities. The numbers in `logits` are the model's raw scores for five candidate words.

▶ **Python code**

```
import math

words  = ["Paris", "Lyon", "Nice", "Rome", "Tokyo"]
logits = [4.0, 3.0, 2.0, 1.0, 0.5]

def probabilities(temp):
    scaled = [x / temp for x in logits]
    exps = [math.exp(x) for x in scaled]
    total = sum(exps)
    return [e / total for e in exps]

for temp in (0.3, 1.0, 2.0):
    probs = probabilities(temp)
    line = "  ".join(f"{w}:{p*100:5.1f}%" for w, p in zip(words, probs))
    print(f"temperature {temp}: {line}")
```

▶ **Output (real output from running this code)**

```
temperature 0.3: Paris: 96.4%  Lyon:  3.4%  Nice:  0.1%  Rome:  0.0%  Tokyo:  0.0%
temperature 1.0: Paris: 63.2%  Lyon: 23.2%  Nice:  8.5%  Rome:  3.1%  Tokyo:  1.9%
temperature 2.0: Paris: 42.2%  Lyon: 25.6%  Nice: 15.5%  Rome:  9.4%  Tokyo:  7.3%
```

TIP: Beginner rule

Use **temperature 0 to 0.3** for tasks with one right answer (code, facts, resume scoring). Use **0.7 to 1.0** for creative tasks (stories, slogans, brainstorming).

1.8 API-Based AI

DEFINITION: API

An **API** (Application Programming Interface) is a doorway that lets one program talk to another. With an **AI API**, your Python program sends a prompt to the provider's servers and receives the generated answer.



You send text and settings; you receive generated text. No GPU needed on your laptop.

Figure 10: How your program talks to an AI model

Why use an API instead of running the model yourself?

	API-based AI	Running a model locally
Hardware	Any laptop works	Needs a strong GPU and lots of RAM
Setup	Install a library, add a key	Download large model files, configure software
Quality	Access to the most capable models	Smaller models, usually less capable
Cost	Pay per token	Free to run after buying hardware
Privacy	Data is sent to the provider	Data stays on your machine

Common AI API providers

Provider	Python library	Typical use
Anthropic (Claude)	`anthropic`	Chat, writing, coding, analysis
OpenAI (GPT)	`openai`	Chat, images, speech
Google (Gemini)	`google-genai`	Chat, multimodal tasks

NOTE: Which provider in this guide?

All code in this guide uses the **Anthropic Python library**. The ideas (keys, messages, parameters, errors, history) are the same for every provider; only small details such as function names change.

Part 2: Prompt Engineering

2.1 What is Prompt Engineering?

DEFINITION: Prompt Engineering

Prompt engineering is the skill of writing clear, well-structured inputs so that an AI model gives accurate, useful and well-formatted outputs. It is like giving instructions to a very capable assistant who has never met you: the clearer your instructions, the better the work.

Ingredients of a good prompt

ROLE	You are a career coach for engineering students.
TASK	Review the resume below and list improvements.
CONTEXT	The student is applying for a Python developer internship.
RULES	Be polite. Maximum 5 points. No more than 15 words each.
FORMAT	Reply as a numbered list.

Figure 11: Five building blocks of a strong prompt (use the ones your task needs)

Before and after

Weak prompt	Strong prompt	Why it is better
"Write about dogs."	"Write a 100-word paragraph for Class 5 students about why dogs are called man's best friend. Use simple words."	States length, audience, topic and style
"Fix my code."	"This Python function should return the average of a list but raises ZeroDivisionError for an empty list. Fix it and explain in 2 lines."	Describes the problem and the expected output
"Resume feedback"	"Act as a recruiter. Give 3 strengths and 3 weaknesses of the resume below for a data-analyst role."	Gives a role, a count and a goal

TIP: The golden rule

If a human would need to ask you three follow-up questions to do the task, your prompt is missing information. Put the answers in the prompt.

2.2 Role Prompting

DEFINITION: Role prompting

Role prompting means telling the AI **who it should act as** (a teacher, a recruiter, a doctor's note summariser). The role changes the vocabulary, depth, tone and focus of the answer.

In chat APIs the role is normally placed in the **system prompt**, so it stays active for the whole conversation.

Role in system prompt	Question: "What is a variable?"	Style of answer
You are a patient Python tutor for school students.	A variable is like a labelled box where you keep a value, e.g. age = 15.	Simple, friendly, with an analogy

Role in system prompt	Question: "What is a variable?"	Style of answer
You are a senior software engineer.	A variable is a named reference to an object in memory; Python is dynamically typed.	Technical, precise
You are a pirate who teaches coding.	Arr! A variable be a treasure chest with a name painted on it, matey!	Fun, playful (creative)

Code: the same question with different roles

▶ **Python code**

```
import anthropic
client = anthropic.Anthropic()          # reads ANTHROPIC_API_KEY automatically

def ask(role, question):
    response = client.messages.create(
        model="claude-sonnet-5-5",
        max_tokens=150,
        system=role,                    # <-- the role goes here
        messages=[{"role": "user", "content": question}],
    )
    return response.content[0].text

q = "What is a variable?"
print(ask("You are a patient Python tutor for school students.", q))
print("-" * 40)
print(ask("You are a senior software engineer. Be precise and technical.", q))
```

▶ **Sample output (wording will vary)**

```
A variable is like a labelled box. You put a value inside it and use the
label to find it later. Example: age = 15 stores the number 15 in "age".
----------------------------------------
A variable is a name bound to an object in memory. Python is dynamically
typed, so the type belongs to the object, not to the variable name.
```

2.3 Zero-Shot and Few-Shot Prompting



Figure 12: Zero-shot versus few-shot

DEFINITION: Zero-shot prompting
You give the instruction **without any examples**. The model relies only on what it already knows.

Example: "Classify the sentiment of this review as Positive, Negative or Neutral: 'The delivery was late but the food was great.'"

DEFINITION: Few-shot prompting
You include **a few solved examples** inside the prompt so the model copies the pattern. This is powerful when you want a very specific format or label set.

Few-shot example (sentiment labels)

▶ **Few-shot prompt**

```
Classify each review as Positive, Negative or Neutral.

Review: "I loved the camera quality!"        -> Positive
Review: "The phone stopped working in a week." -> Negative
Review: "It arrived on Monday."               -> Neutral

Review: "Battery life is amazing but the screen scratches easily." ->
```

▶ **Sample output**

```
Neutral  (mixed review)
```

Few-shot in Python: examples as message history

With chat APIs, a neat way to give examples is to write them as earlier **user / assistant** messages. The model sees them as the pattern to follow. The program below only *builds* the messages, so you can run it without an API key.

▶ **Python code**

```
import json

examples = [
    ("I loved the camera quality!", "Positive"),
    ("The phone stopped working in a week.", "Negative"),
    ("It arrived on Monday.", "Neutral"),
]

def build_few_shot(new_review):
    messages = []
    for review, label in examples:
        messages.append({"role": "user", "content": review})
        messages.append({"role": "assistant", "content": label})
    messages.append({"role": "user", "content": new_review})
    return messages

msgs = build_few_shot("Great sound but the case feels cheap.")
print("Total messages:", len(msgs))
print(json.dumps(msgs[-3:], indent=2))
```

▶ **Output (real output from running this code)**

```
Total messages: 7
[
  {
    "role": "user",
    "content": "It arrived on Monday."
  },
  {
    "role": "assistant",
    "content": "Neutral"
  },
  {
    "role": "user",
    "content": "Great sound but the case feels cheap."
  }
]
```

When to use	Zero-shot	Few-shot
Simple, common tasks (summarise, translate)	Yes	Not needed
Custom labels or unusual format	Risky	Best choice
Want fewer tokens / lower cost	Best choice	Costs more
Need consistent output across many inputs	Sometimes	Best choice

2.4 Structured Prompting

DEFINITION: Structured prompting

Structured prompting means writing the prompt in **clearly separated sections** (role, task, context, rules, format) instead of one long paragraph. The model can then find each instruction easily, and so can you when you debug.

A simple framework: R-T-C-F

Letter	Meaning	Example
R	Role - who the AI is	You are an interview coach.
T	Task - what to do	Ask me 3 questions on Python lists.
C	Context / constraints - details and limits	I am a beginner. Keep each question under 20 words.
F	Format - how to answer	Numbered list, then wait for my answers.

Using delimiters

Delimiters are markers such as ###, triple quotes or XML-style tags that separate your instructions from the data. They stop the model from mixing up the two.

▶ **A structured prompt**

```
You are a helpful editor.

### TASK
Correct the grammar of the text between the <text> tags. Do not change the meaning.

### RULES
- Return only the corrected text.
- Keep it under 50 words.

<text>
me and my friend goes to college by bus everyday
</text>
```

▶ **Sample output**

```
My friend and I go to college by bus every day.
```

2.5 Context Injection

DEFINITION: Context injection

Context injection means **inserting extra information into the prompt** - such as a resume, a lecture note, a product FAQ or today's date - so the model can answer using facts it did not learn in training.



The model only 'knows' your document because you pasted it into the prompt.

Figure 13: Your data is injected into the prompt before it reaches the model

An LLM does not know your college timetable, your resume or your company's refund policy. If you paste that text into the prompt and tell the model to *answer only from it*, the answers become accurate

and specific. This idea is the base of many "chat with your PDF" tools.

Example prompt

▶ **Prompt with injected context**

```
Answer the question using ONLY the information in the context.
If the answer is not in the context, say "I don't have that information."

Context:
"""
Library timings: Monday to Saturday, 9 AM to 6 PM. Closed on Sundays.
Books can be borrowed for 14 days. Late fee is Rs. 2 per day.
"""

Question: How much is the fine if I return a book 5 days late?
```

▶ **Sample output**

```
The late fee is Rs. 2 per day, so 5 days late costs Rs. 10.
```

2.6 Output Formatting

When your Python program will *use* the AI's answer (store it, show it in a table, count items), you need predictable output. Tell the model the exact format you want.

Format	Prompt wording	Good for
Bullet list	"Answer in exactly 5 bullet points."	Summaries
Table	"Give a markdown table with columns Skill, Level."	Comparisons
JSON	"Return only valid JSON with keys: name, score, advice."	Programs that read the answer
Word limit	"Maximum 60 words."	Short answers, captions
Steps	"Explain in numbered steps."	Tutorials

JSON output example

▶ **Prompt**

```
Analyse the sentence and return ONLY valid JSON, no extra words.
Keys: "sentiment" (Positive/Negative/Neutral), "confidence" (0 to 1), "reason" (max 10 words).

Sentence: "The teacher explained everything clearly."
```

▶ **Sample output**

```
{"sentiment": "Positive", "confidence": 0.96, "reason": "Praises the teacher's clear explanation"}
```

Reading the JSON in Python

▶ **Python code**

```
import json

ai_reply = '{"sentiment": "Positive", "confidence": 0.96, "reason": "Praises clear teaching"}'

data = json.loads(ai_reply)          # text -> Python dictionary
print("Sentiment :", data["sentiment"])
print("Confidence:", data["confidence"])
print("Type      :", type(data))
```

▶ **Output (real output from running this code)**

```
Sentiment : Positive
Confidence: 0.96
Type      : <class 'dict'>
```

2.7 Prompt Templates

DEFINITION: Prompt template

A **prompt template** is a reusable prompt with **blanks (placeholders)** that your program fills with different values each time. In Python you can build templates with f-strings or `str.format()`.

▶ **Python code**

```
TEMPLATE = """You are a {role}.
Explain the topic "{topic}" to a {level} student.
Use at most {words} words and finish with one practice question."""

prompt = TEMPLATE.format(role="Python tutor", topic="for loops",
                         level="beginner", words=80)
print(prompt)
```

▶ **Output (real output from running this code)**

```
You are a Python tutor.
Explain the topic "for loops" to a beginner student.
Use at most 80 words and finish with one practice question.
```

A template with a function

▶ **Python code**

```
def resume_prompt(resume_text, job_title):
    return (
        f"Act as a recruiter hiring a {job_title}.\n"
        f"Review this resume and give 3 strengths and 3 improvements.\n\n"
        f"RESUME:\n{resume_text}"
    )

print(resume_prompt("B.Tech student, knows Python and SQL.", "Data Analyst Intern"))
```

▶ **Output (real output from running this code)**

```
Act as a recruiter hiring a Data Analyst Intern.
Review this resume and give 3 strengths and 3 improvements.

RESUME:
B.Tech student, knows Python and SQL.
```

REAL-WORLD USE: Why templates matter

One template can power thousands of requests: change *topic* and the same code becomes a quiz maker, a lesson planner or an email writer.

2.8 Prompt Engineering Cheat Sheet

Do	Don't
Be specific about audience, length and format	Write vague one-word prompts
Give examples when the format matters	Assume the model can guess your style
Separate instructions from data with delimiters	Mix pasted text with commands
Test with several inputs and refine	Judge a prompt from one result
Tell the AI what to do	Only say what NOT to do

Practice exercises

1. Rewrite "Explain AI" as a structured prompt for Class 8 students using the R-T-C-F framework.
2. Create a few-shot prompt that classifies messages as *Urgent* or *Not urgent* using three examples.
3. Write a template with placeholders *{subject}* and *{count}* that asks for MCQ questions in JSON.

Part 3: Python + AI APIs

3.1 How an API Request Works

Every AI call is a web request. Your program (the **client**) sends a request over the internet using **HTTP**. The AI provider's computer (the **server**) runs the model and sends back a response. Libraries such as `anthropic` hide the HTTP details so you can write normal Python.

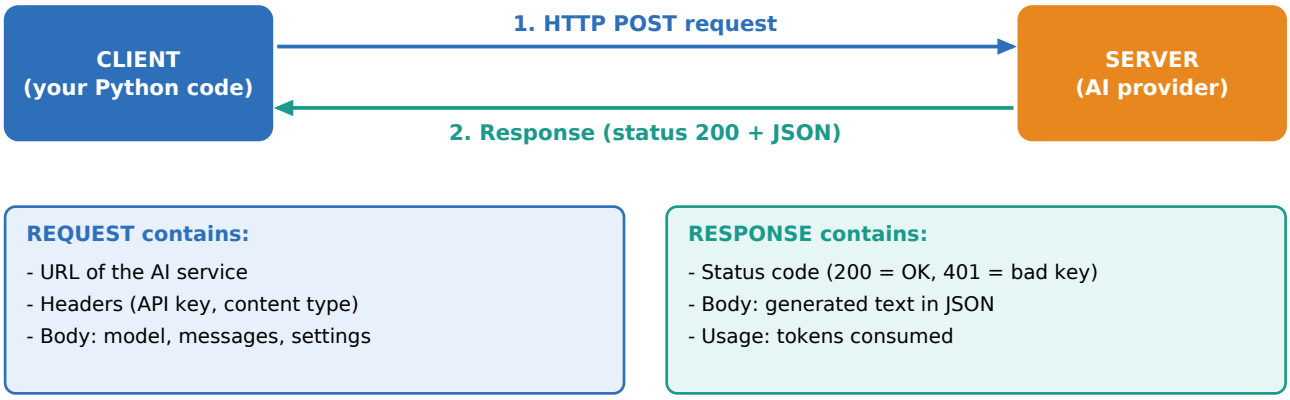


Figure 14: One request and one response

What the request body looks like (JSON)

▶ **JSON request body (the library builds this for you)**

```
{
  "model": "claude-sonnet-5-5",
  "max_tokens": 200,
  "system": "You are a helpful tutor.",
  "messages": [
    {"role": "user", "content": "What is a Python list?"}
  ]
}
```

DEFINITION: JSON

JSON (JavaScript Object Notation) is a text format for sending data. It looks exactly like Python dictionaries and lists, so Python can convert between the two easily.

3.2 API Keys

DEFINITION: API key

An **API key** is a long secret string (like a password) that proves to the provider that the request comes from **your account**. The provider uses it to allow access and to bill you.

How to get a key

1. Create an account on your AI provider's developer console.
2. Open the **API keys** page and click *Create key*.
3. Copy the key immediately; many consoles show it only once.
4. Store it safely (next section) - never inside your Python file.

Golden rules for keys

Do	Never
Keep the key in a `.env` file or environment variable	Write the key directly in your code: `key = "sk-abc..."`
Add `.env` to `.gitignore`	Upload the key to GitHub, Google Drive or a screenshot
Create a separate key for each project	Share your key with friends or in WhatsApp groups
Delete (revoke) a key at once if it leaks	Ignore a leaked key - strangers can spend your money
Set a spending limit in the console	Leave a key without any usage limit

WARNING: Bad example

`client = anthropic.Anthropic(api_key="sk-ant-12345")`
If you push this line to a public GitHub repository, automated bots can find the key within minutes.

EXAMPLE: Good example

`client = anthropic.Anthropic()` - the library automatically reads the key from the environment variable `ANTHROPIC_API_KEY`.

3.3 Environment Variables

DEFINITION: Environment variable

An **environment variable** is a named value stored *outside* your program, in the operating system's memory. Your code reads it by name, so the secret value never appears in the source file.

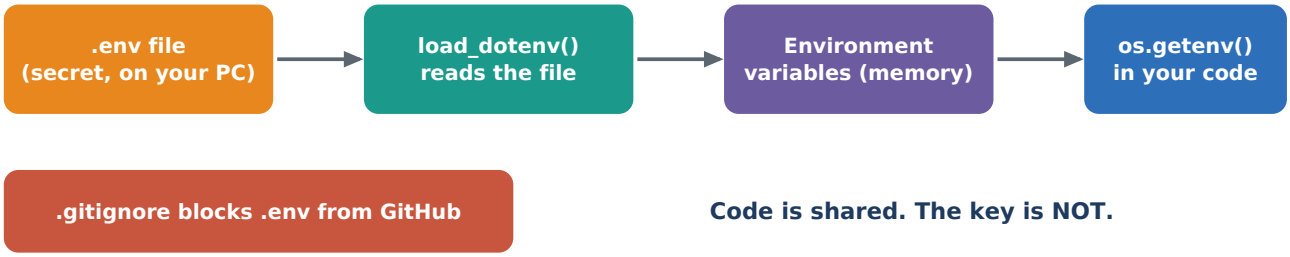


Figure 15: The safe path of a secret from file to program

Reading variables in Python

▶ **Python code**

```
import os

os.environ["DEMO_KEY"] = "abc123"          # we set one just for this demo

print(os.getenv("DEMO_KEY"))                # exists
print(os.getenv("MISSING_KEY"))             # does not exist -> None
print(os.getenv("MISSING_KEY", "default"))  # fallback value
```

▶ **Output (real output from running this code)**

```
abc123
None
default
```

Using a .env file (recommended)

Create a file named exactly `.env` in your project folder and put one line in it:

▶ **.env file (no quotes, no spaces around =)**

```
ANTHROPIC_API_KEY=your-secret-key-goes-here
```

▶ **Python code**

```
# pip install python-dotenv
import os
from dotenv import load_dotenv

load_dotenv()                                    # loads the .env file
api_key = os.getenv("ANTHROPIC_API_KEY")

if api_key is None:
    print("Key not found. Did you create the .env file?")
else:
    print("Key loaded. Length:", len(api_key))   # never print the key itself!
```

▶ **Sample output (your key length may differ)**

```
Key loaded. Length: 108
```

TIP: .gitignore

Create a file named `.gitignore` and add the line `.env`. Git will then never upload your secrets.

3.4 Sending Your First Request

Step 1: Set up the project

▶ **Folder structure**

```
my_ai_assistant/
|-- .env               <- your secret key
|-- .gitignore         <- contains the line  .env
|-- first_request.py   <- today's first program
```

▶ **Terminal - install the libraries**

```
pip install anthropic python-dotenv
```

Step 2: Write the program

▶ **first_request.py**

```
from dotenv import load_dotenv
import anthropic

load_dotenv()
client = anthropic.Anthropic()          # finds ANTHROPIC_API_KEY itself

response = client.messages.create(
    model="claude-sonnet-5-5",          # check the docs for current model names
    max_tokens=200,                     # reply length limit
    messages=[
        {"role": "user", "content": "Explain what an API is in two sentences."}
    ],
)

print(response.content[0].text)
```

▶ **Sample output**

```
An API is a set of rules that lets one program ask another program for data
or services. It works like a waiter: you place an order, the waiter takes it
to the kitchen, and brings back your food.
```

Line-by-line explanation

Line	What it does
`load_dotenv()`	Reads the .env file into environment variables
`anthropic.Anthropic()`	Creates the client object that sends requests
`client.messages.create(...)`	Sends the request and waits for the answer
`messages=[...]`	The conversation so far, as a list of dictionaries
`response.content[0].text`	Extracts the generated text from the response

3.5 Receiving and Reading Responses

The API returns a **response object**. You usually need only the text, but it also carries useful details.

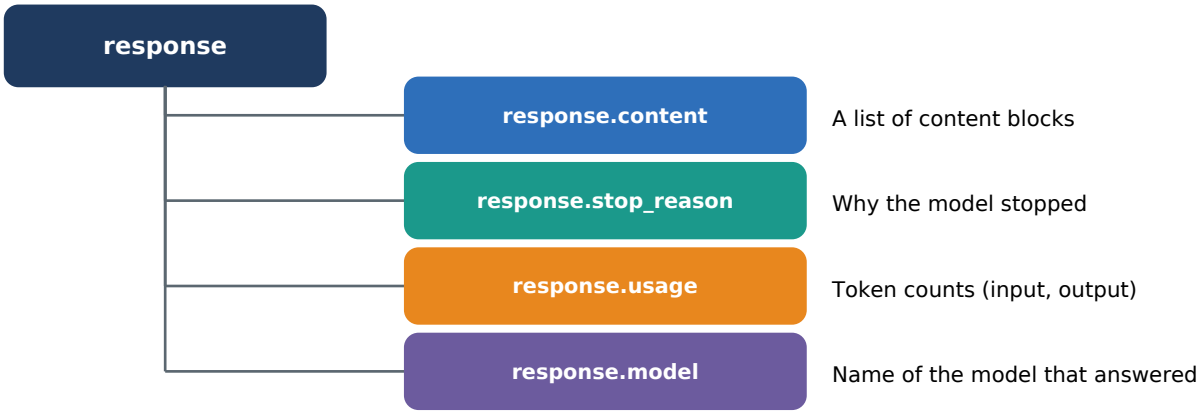


Figure 16: Important parts of the response object

Looking inside a response

▶ **Python code**

```
print("Answer     :", response.content[0].text)
print("Model      :", response.model)
print("Stop reason:", response.stop_reason)
print("Input tokens :", response.usage.input_tokens)
print("Output tokens:", response.usage.output_tokens)
```

▶ **Sample output**

```
Answer     : An API is a set of rules that lets programs talk to each other.
Model      : claude-sonnet-5-5
Stop reason: end_turn
Input tokens : 16
Output tokens: 58
```

stop_reason	Meaning	What to do
end_turn	The model finished naturally	Nothing - all good
max_tokens	Reply was cut because it hit your limit	Increase `max_tokens` or ask for a shorter answer
stop_sequence	A stop marker you defined appeared	Expected behaviour

TIP: Track your usage

Print usage during development. It shows exactly how many tokens each call costs and helps you notice when the history is getting too long.

3.6 Error Handling

Real programs must expect failure: the Wi-Fi drops, the key is wrong, or too many requests are sent. **Error handling** means catching these problems and responding politely instead of crashing.

Problem	Status	Python exception (anthropic library)	Typical fix
Wrong or missing key	401	AuthenticationError	Check .env file
Too many requests	429	RateLimitError	Wait, then retry
No internet / timeout	-	APIConnectionError	Check network, retry
Invalid request (bad model name, empty message)	400	BadRequestError	Fix your parameters
Provider problem	500+	APIStatusError	Retry later

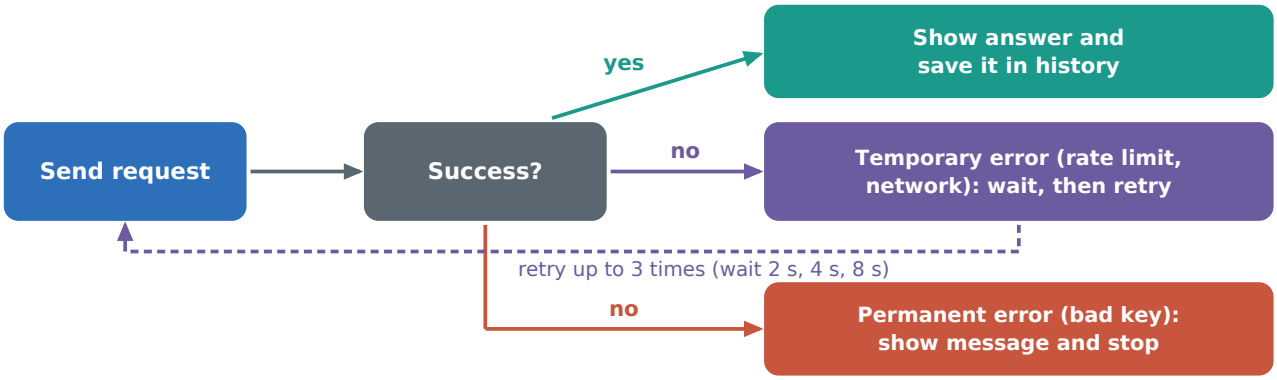


Figure 17: Decision flow for a failed request

try / except around an API call

▶ **Python code**

```
import anthropic

client = anthropic.Anthropic()

def safe_ask(question):
    try:
        response = client.messages.create(
            model="claude-sonnet-5-5",
            max_tokens=200,
            messages=[{"role": "user", "content": question}],
        )
        return response.content[0].text

    except anthropic.AuthenticationError:
        return "[Error] Invalid API key. Check your .env file."
    except anthropic.RateLimitError:
        return "[Error] Too many requests. Please wait a moment."
    except anthropic.APIConnectionError:
        return "[Error] Cannot reach the server. Check your internet."
    except anthropic.APIStatusError as err:
        return f"[Error] Server returned status {err.status_code}."
```

Retry with exponential backoff

For temporary errors, wait a little and try again - waiting longer each time (2 s, then 4 s, then 8 s). This is called **exponential backoff**. The demo below uses a fake function that fails twice, so you can see the idea without an API key.

▶ **Python code**

```
import time

calls = {"count": 0}

def flaky_api():
    calls["count"] += 1
    if calls["count"] < 3:
        raise ConnectionError("network down")
    return "Hello from the AI!"

def call_with_retry(max_tries=4):
    for attempt in range(1, max_tries + 1):
        try:
            return flaky_api()
        except ConnectionError as err:
            wait = 2 ** attempt
            print(f"Attempt {attempt} failed ({err}). Would wait {wait}s")
            time.sleep(0.01)          # tiny pause so this demo runs fast
    return "Gave up."

print("Result:", call_with_retry())
```

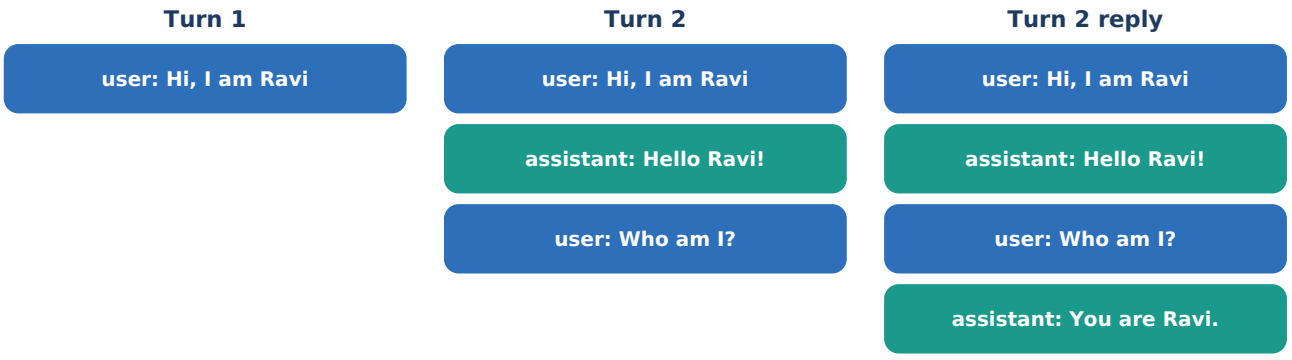
▶ **Output (real output from running this code)**

```
Attempt 1 failed (network down). Would wait 2s
Attempt 2 failed (network down). Would wait 4s
Result: Hello from the AI!
```

3.7 Conversation History

DEFINITION: Conversation history

Conversation history is the **list of all earlier messages** (user and assistant) that your program stores and sends again with every new request. Because the API is *stateless* (it forgets everything after each call), this list is the only memory the AI has.



The whole list is sent every time, so the AI can 'remember'.

Figure 18: The history list grows after every turn

The pattern in code is always the same: **(1) append the user message, (2) send the list, (3) append the AI reply**.

▶ **Python code**

```
history = []

def fake_ai(messages):
    """Stands in for the real API so we can test the logic offline."""
    last = messages[-1]["content"]
    if "who am i" in last.lower():
        known = any("ravi" in m["content"].lower() for m in messages[:-1])
        return "You are Ravi." if known else "I don't know your name."
    return "Hello! Nice to meet you."

def chat(user_text):
    history.append({"role": "user", "content": user_text})      # step 1
    reply = fake_ai(history)                                     # step 2
    history.append({"role": "assistant", "content": reply})     # step 3
    return reply

print("AI:", chat("Hi, I am Ravi"))
print("AI:", chat("Who am I?"))
print("Messages stored:", len(history))
```

▶ **Output (real output from running this code)**

```
AI: Hello! Nice to meet you.
AI: You are Ravi.
Messages stored: 4
```

Managing long conversations

The history grows forever, but the context window and your budget do not. Three common solutions:

Method	How it works	Pros	Cons
Sliding window	Keep only the last N messages	Very simple	Forgets early facts
Summarise	Ask the AI to summarise old messages into 1 short note	Keeps key facts	Extra API call
Reset	Clear history for a new topic	Cheapest	Loses everything



Figure 19: Sliding window keeps only the newest messages

▶ **Python code**

```
def trim_history(history, max_messages=4):
    if len(history) <= max_messages:
        return history
    history = history[-max_messages:]
    while history and history[0]["role"] != "user":   # must start with user
        history = history[1:]
    return history

history = [{"role": "user" if i % 2 == 0 else "assistant",
            "content": f"message {i + 1}"} for i in range(8)]

trimmed = trim_history(history, 4)
print([m["content"] for m in trimmed])
```

▶ **Output (real output from running this code)**

```
['message 5', 'message 6', 'message 7', 'message 8']
```

Part 4: AI Applications

Everything you learned so far - roles, templates, history, error handling - combines into real applications. This part shows five popular ones. Each application is mostly a **good system prompt + a template + the same API call**.

4.1 AI Chatbots

DEFINITION: Chatbot

A **chatbot** is a program that holds a conversation with a user in natural language. An AI chatbot uses an LLM to understand the question and write the reply.

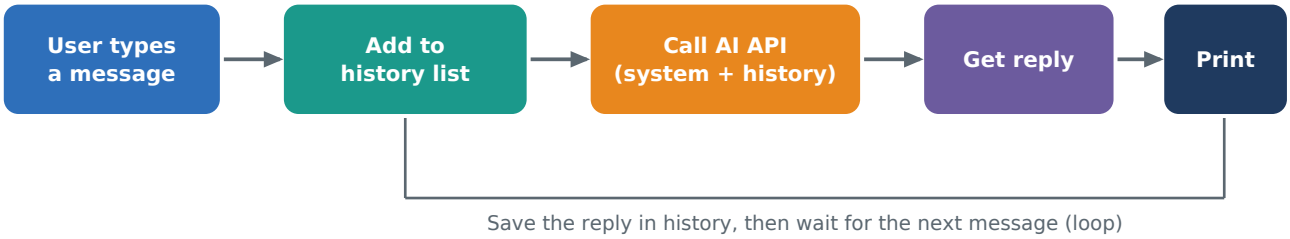


Figure 20: The chatbot loop - the heart of your Day 4 project

Types of chatbots

Type	Example	Key ingredient
Customer support	"Where is my order?"	Context injection of FAQs and policies
Personal assistant	Reminders, email drafts	Role prompt + conversation history
Domain expert	Legal or medical information helper	Careful rules and disclaimers
Entertainment	Story or game characters	High temperature, strong persona

REAL-WORLD USE: Where you see them

Bank websites, food-delivery apps, college admission portals, telecom help desks, and WhatsApp business accounts.

TIP: A good chatbot system prompt

State the role, the tone, the limits ("If you are not sure, say so"), and the reply length. Example: "*You are CollegeBot for STDN College. Answer only about admissions, fees and timetables. Reply in under 60 words. If the answer is unknown, ask the user to contact the office.*"

4.2 Resume Analysis

DEFINITION: Resume analysis

Using AI to read a resume (and optionally a job description) and return **strengths, missing skills, and suggestions**. Recruiters and students both use this idea.



Output: score, skills found, missing skills, advice

Figure 21: Resume analysis pipeline

Step 1: the prompt template

▶ **Prompt template with placeholders**

```
RESUME_PROMPT = """You are an experienced technical recruiter.
Compare the RESUME with the JOB DESCRIPTION.

Return ONLY valid JSON with these keys:
  "match_score"    : number from 0 to 100,
  "skills_found"   : list of skills present in both,
  "skills_missing" : list of required skills not in the resume,
  "advice"         : list of exactly 3 short improvement tips

<resume>
{resume}
</resume>

<job_description>
{job}
</job_description>"""
```

Step 2: read the AI's JSON answer and show a clean report

The AI reply below is typed by hand so the program runs offline. With a real call, `ai_reply` would be `response.content[0].text`.

▶ **Python code**

```
import json

ai_reply = """{
  "match_score": 72,
  "skills_found": ["Python", "SQL"],
  "skills_missing": ["Power BI", "Statistics"],
  "advice": ["Add one dashboard project", "List your internship results",
             "Mention statistics coursework"]
}"""

try:
    report = json.loads(ai_reply)
except json.JSONDecodeError:
    print("The AI did not return valid JSON. Try again.")
else:
    print(f"Match score : {report['match_score']}/100")
    print("Found       :", ", ".join(report["skills_found"]))
    print("Missing     :", ", ".join(report["skills_missing"]))
    print("Advice:")
    for i, tip in enumerate(report["advice"], 1):
        print(f"  {i}. {tip}")
```

▶ **Output (real output from running this code)**

```
Match score : 72/100
Found       : Python, SQL
Missing     : Power BI, Statistics
Advice:
  1. Add one dashboard project
  2. List your internship results
  3. Mention statistics coursework
```

WARNING: Responsible use

AI resume scores are **suggestions, not final judgements**. A model can be biased or miss context. Never use it as the only reason to reject a person.

4.3 Content Generation

DEFINITION: Content generation

Using an LLM to **draft new text**: emails, social-media posts, product descriptions, lesson plans, reports, or code comments. The human then reviews and edits the draft.

The secret is a template with **audience, tone, length and purpose** as placeholders.

▶ **Python code**

```
CONTENT_TEMPLATE = (
    "Write a {kind} about {topic}.\n"
    "Audience: {audience}\n"
    "Tone: {tone}\n"
    "Length: about {words} words.\n"
    "End with a call to action."
)

jobs = [
    dict(kind="LinkedIn post", topic="finishing my first AI project",
         audience="recruiters", tone="professional and humble", words=60),
    dict(kind="email", topic="requesting a project extension",
         audience="my professor", tone="polite and respectful", words=80),
]

for job in jobs:
    print(CONTENT_TEMPLATE.format(**job))
    print("-" * 45)
```

▶ **Output (real output from running this code)**

```
Write a LinkedIn post about finishing my first AI project.
Audience: recruiters
Tone: professional and humble
Length: about 60 words.
End with a call to action.
---------------------------------------------
Write a email about requesting a project extension.
Audience: my professor
Tone: polite and respectful
Length: about 80 words.
End with a call to action.
---------------------------------------------
```

Use case	Temperature	Tip
Formal email / report	0.2 - 0.4	Give facts; ask for a polite tone
Blog or social post	0.6 - 0.8	Give a style example (few-shot)
Slogans, names, ideas	0.9 - 1.0	Ask for 10 options, then pick the best

NOTE: Always review

AI drafts can include wrong facts or a generic tone. Treat the output as a first draft, check every fact, and add your own voice.

4.4 Study Assistants

DEFINITION: Study assistant

An AI tutor that **explains topics, creates quizzes, summarises notes and makes flashcards** - available 24 hours a day and patient with repeated questions.

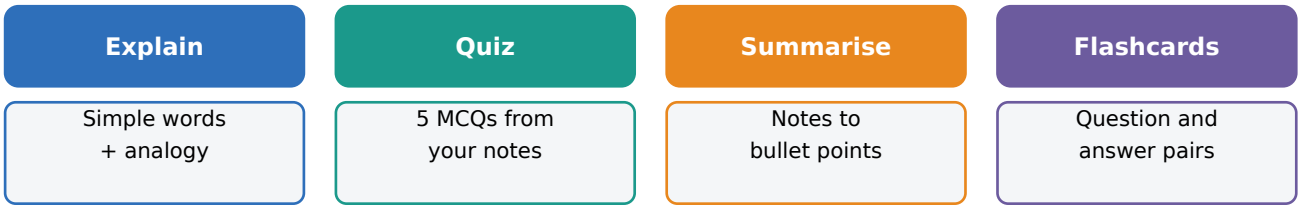


Figure 22: Four modes of a study assistant

One program, four modes

▶ **Python code**

```
MODES = {
    "explain":   "Explain '{text}' in simple words with one real-life analogy.",
    "quiz":      "Create 3 multiple-choice questions (with answers) from: {text}",
    "summarise": "Summarise the following notes in 4 bullet points: {text}",
    "flashcards":"Make 3 flashcards (Q and A) from: {text}",
}

def study_prompt(mode, text):
    return MODES[mode].format(text=text)

notes = "Photosynthesis converts light energy into chemical energy in plants."
for mode in ("explain", "quiz"):
    print(f"[{mode.upper()}]", study_prompt(mode, notes))
```

▶ **Output (real output from running this code)**

```
[EXPLAIN] Explain 'Photosynthesis converts light energy into chemical energy in plants.' in simple words with one real-
[QUIZ] Create 3 multiple-choice questions (with answers) from: Photosynthesis converts light energy into chemical energ
```

Study-assistant system prompt

▶ **Example system prompt**

```
You are a patient tutor for first-year engineering students.
- Explain step by step in simple English.
- Use one everyday analogy.
- After explaining, ask ONE question to check understanding.
- If the student is wrong, hint first; give the answer only if they ask.
```

REAL-WORLD USE: Applications

Exam revision, homework doubts, language practice, coding practice, and turning lecture notes into flashcards.

4.5 Interview Preparation

DEFINITION: AI interview coach

An assistant that plays the **interviewer**: it asks one question at a time, listens to your answer, gives feedback and a score, then asks the next question.

System prompt for the interviewer

▶ **Interviewer prompt**

```
You are a professional but friendly interviewer for a Python Developer
Intern role.
Rules:
1. Ask ONE question at a time and wait for my answer.
2. After my answer, give: a score out of 10, one strength, one improvement.
3. Then ask the next question. Increase the difficulty slowly.
4. After 5 questions, give a final summary and overall score.
```

Sample interview

▶ **Sample output**

```
AI : Question 1: What is the difference between a list and a tuple in Python?
You: A list can be changed but a tuple cannot be changed.
AI : Score: 7/10. Strength: correct core idea (mutable vs immutable).
     Improve: add an example, e.g. tuples can be dictionary keys.
     Question 2: What does the 'self' keyword mean in a class?
```

Tracking scores in Python

▶ **Python code**

```
scores = []

def record(question, score):
    scores.append((question, score))

record("List vs tuple", 7)
record("What is self?", 8)
record("Explain a dictionary", 6)

total = sum(score for _, score in scores)
average = total / len(scores)
print(f"Questions answered: {len(scores)}")
print(f"Average score     : {average:.1f}/10")
print("Weakest topic     :", min(scores, key=lambda x: x[1])[0])
```

▶ **Output (real output from running this code)**

```
Questions answered: 3
Average score     : 7.0/10
Weakest topic     : Explain a dictionary
```

TIP: Practice routine

Do a 10-minute mock interview daily. Change the role in the system prompt (data analyst, web developer, HR round) to practise different interviews.

Part 5: Hands-On Project - Python AI Assistant

5.1 Project Goal

Build a **console chatbot** in Python that uses everything from today. The final program has these requirements:

Requirement	How we meet it
User input	`input()` inside a `while True` loop
AI responses	`client.messages.create()` with a system prompt
Conversation history	A list of role/content dictionaries sent with every request
Error handling	try / except for key, network, rate-limit and server errors, with retries
Secure key	`.env` file + `os.getenv()`
Extras	Personas (tutor, interviewer, resume coach), /save, /reset, token usage display

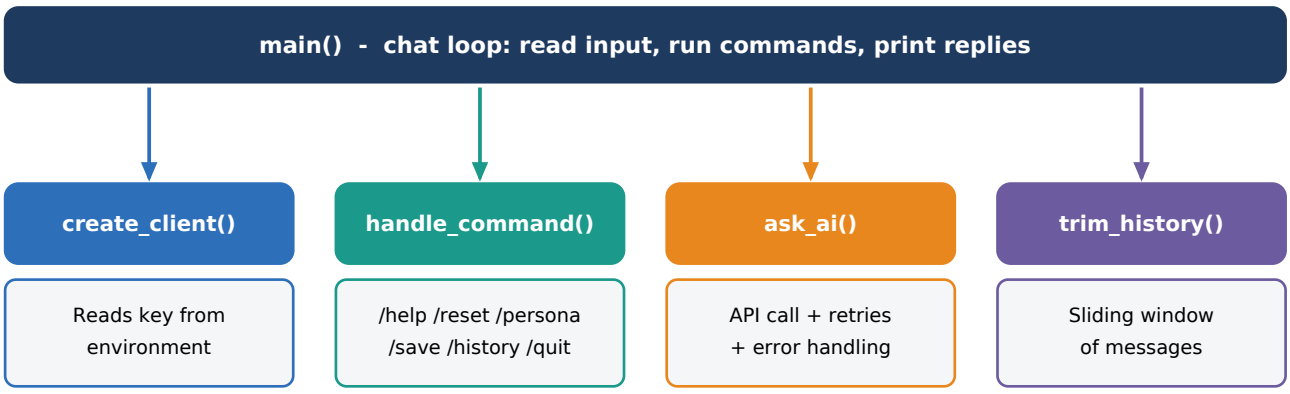


Figure 23: Program structure - five small functions working together

Folder structure

▶ **Project files**

```
ai_assistant/
|-- .env              <- ANTHROPIC_API_KEY=your-key
|-- .gitignore        <- .env
|-- ai_assistant.py   <- the program (next pages)
|-- chat_log.txt      <- created when you type /save
```

5.2 The Complete Code

The program is shown in five sections. Type them one after another into a single file named `ai_assistant.py`.

Section 1: imports and settings

▶ **ai_assistant.py - part 1 of 5**

```
"""Day 4 Project - Python AI Assistant (console chatbot)."""
import os
import sys
import time

import anthropic
from dotenv import load_dotenv

load_dotenv()                       # reads the .env file into environment variables

MODEL = "claude-sonnet-5-5"         # check the docs for the latest model name
MAX_TOKENS = 500                    # maximum length of each reply
TEMPERATURE = 0.7                   # 0 = focused, 1 = creative
MAX_MESSAGES = 20                   # how many past messages we keep
MAX_RETRIES = 3                     # how many times we retry a failed call

PERSONAS = {
    "general": "You are a helpful and concise AI assistant.",
    "tutor": (
        "You are a friendly Python tutor for beginners. Explain in simple "
        "words, give one short example, and end with one practice question."
    ),
    "interview": (
        "You are a kind but professional technical interviewer. Ask ONE "
        "question at a time. After each answer give short feedback and a "
        "score out of 10, then ask the next question."
    ),
    "resume": (
        "You are a career coach. When the user pastes a resume, reply with "
        "three headings: Strengths, Weaknesses, Improvements."
    ),
}

HELP_TEXT = """Commands:
  /help            show this message
  /persona NAME    switch role (general, tutor, interview, resume)
  /history         show how many messages are remembered
  /save            save the chat to chat_log.txt
  /reset           forget the conversation and start again
  /quit            exit the program"""
```

Name	Purpose
MAX_TOKENS, TEMPERATURE	Reply length and creativity
MAX_MESSAGES	Size of the sliding window for history
PERSONAS	Dictionary of system prompts; the user can switch between them

Section 2: client and history helpers

▶ **ai_assistant.py - part 2 of 5**

```
def create_client():
    """Create the API client using the key stored in the environment."""
    api_key = os.getenv("ANTHROPIC_API_KEY")
    if not api_key:
        print("ERROR: ANTHROPIC_API_KEY not found. Add it to your .env file.")
        sys.exit(1)
    return anthropic.Anthropic(api_key=api_key)

def trim_history(history, max_messages=MAX_MESSAGES):
    """Keep only the most recent messages (a sliding window)."""
    if len(history) <= max_messages:
        return history
    history = history[-max_messages:]
    while history and history[0]["role"] != "user":   # must start with user
        history = history[1:]
    return history

def save_chat(history, filename="chat_log.txt"):
    """Write the conversation to a text file."""
    with open(filename, "w", encoding="utf-8") as file:
        for message in history:
            speaker = "You" if message["role"] == "user" else "AI"
            file.write(f"{speaker}: {message['content']}\n\n")
    return filename
```

create_client() stops the program with a helpful message if the key is missing. **trim_history()** is the sliding window you met in Part 3. **save_chat()** writes the conversation to a text file.

Section 3: talking to the AI with error handling

▶ **ai_assistant.py - part 3 of 5**

```
def ask_ai(client, history, system_prompt):
    """Send the conversation to the AI. Returns (text, usage) or (None, None)."""
    for attempt in range(1, MAX_RETRIES + 1):
        try:
            response = client.messages.create(
                model=MODEL,
                max_tokens=MAX_TOKENS,
                temperature=TEMPERATURE,
                system=system_prompt,
                messages=history,
            )
            return response.content[0].text, response.usage

        except anthropic.AuthenticationError:
            print("[Error] Your API key is invalid. Check your .env file.")
            return None, None

        except (anthropic.RateLimitError, anthropic.APIConnectionError) as err:
            wait = 2 ** attempt                      # 2, 4, 8 seconds
            print(f"[Warning] {type(err).__name__}. Retry {attempt}/"
                  f"{MAX_RETRIES} in {wait}s...")
            time.sleep(wait)

        except anthropic.APIStatusError as err:
            print(f"[Error] The server returned status {err.status_code}.")
            return None, None

    print("[Error] Still failing after several tries. Please try later.")
    return None, None
```

NOTE: How the retry loop works

`for attempt in range(1, MAX_RETRIES + 1)` tries up to 3 times. Temporary errors (rate limit, network) wait 2, 4, then 8 seconds. Permanent errors (bad key, server status errors) return `(None, None)` immediately. The caller checks for None and removes the failed user message from history.

Section 4: command handler

▶ **ai_assistant.py - part 4 of 5**

```
def handle_command(command, state):
    """Run a /command. Returns False when the program should exit."""
    parts = command.split()
    name = parts[0].lower()

    if name == "/quit":
        return False
    if name == "/help":
        print(HELP_TEXT)
    elif name == "/reset":
        state["history"] = []
        print("Conversation cleared.")
    elif name == "/history":
        print(f"Remembered messages: {len(state['history'])}")
    elif name == "/save":
        print(f"Saved to {save_chat(state['history'])}")
    elif name == "/persona":
        if len(parts) > 1 and parts[1] in PERSONAS:
            state["persona"] = parts[1]
            state["history"] = []
            print(f"Persona changed to '{parts[1]}'. New conversation started.")
        else:
            print("Choose one of:", ", ".join(PERSONAS))
    else:
        print("Unknown command. Type /help.")
    return True
```

Section 5: the main chat loop

▶ **ai_assistant.py - part 5 of 5**

```
def main():
    client = create_client()
    state = {"history": [], "persona": "general"}

    print("=" * 50)
    print(" Python AI Assistant  (type /help for commands)")
    print("=" * 50)

    while True:
        try:
            user_input = input("\nYou: ").strip()
        except (KeyboardInterrupt, EOFError):
            print("\nGoodbye!")
            break

        if not user_input:
            continue
        if user_input.startswith("/"):
            if not handle_command(user_input, state):
                print("Goodbye!")
                break
            continue

        state["history"].append({"role": "user", "content": user_input})
        state["history"] = trim_history(state["history"])

        reply, usage = ask_ai(client, state["history"],
                              PERSONAS[state["persona"]])
        if reply is None:
            state["history"].pop()          # remove the message that failed
            continue

        state["history"].append({"role": "assistant", "content": reply})
        print(f"\nAI: {reply}")
        print(f"   [tokens used: {usage.input_tokens} in, "
              f"{usage.output_tokens} out]")

if __name__ == "__main__":
    main()
```

Flow of one chat turn

1. Read the user's text with `input()`; ignore empty lines.
2. If it starts with `/`, run the command and ask for the next input.
3. Otherwise append it to history as a *user* message and trim the history.
4. Call `ask_ai()` with the history and the persona's system prompt.
5. If it failed, remove the user message and continue; if it worked, append the reply as an *assistant* message and print it.

5.3 Running the Program

▶ **Terminal**

```
python ai_assistant.py
```

▶ **Sample session (AI wording will differ in your run)**

```
================================================
 Python AI Assistant  (type /help for commands)
================================================

You: Hi, my name is Ravi and I am learning Python.

AI: Hi Ravi! Great to meet you. Python is a wonderful first language.
    What would you like to learn today?
   [tokens used: 38 in, 31 out]

You: What is my name?

AI: Your name is Ravi.
   [tokens used: 84 in, 8 out]

You: /history
Remembered messages: 4

You: /save
Saved to chat_log.txt

You: /quit
Goodbye!
```

NOTE: See the memory working

In the second turn the AI knew the name "Ravi" only because the first turn was still in the history list. Notice the *tokens in* number grew from 38 to 84 - the history is being re-sent.

Testing checklist

Test	Expected result
Delete the key from .env and run	Friendly 'key not found' message, no crash
Put a wrong key in .env	'Your API key is invalid' message
Turn off Wi-Fi and send a message	Retry warnings, then a polite failure message
Press Enter on an empty line	Nothing happens; prompt appears again
Type /persona pirate	List of valid personas is shown
Chat for more than 20 messages	No crash; oldest messages are dropped

5.4 Troubleshooting

Symptom	Likely cause	Fix
ModuleNotFoundError: anthropic	Library not installed	`pip install anthropic python-dotenv`
Key not found message	.env missing or in wrong folder	Keep .env in the same folder you run Python from
401 / invalid key	Typing mistake or revoked key	Create a new key and paste it again
Error mentioning model name	Model name outdated	Copy the current name from the provider docs
Reply stops in mid-sentence	max_tokens too low	Increase MAX_TOKENS to 800 or more
Error mentioning a parameter	Model does not accept it	Remove that parameter (e.g. temperature) and retry

5.5 Challenges to Extend the Project

Level 1 - Easy

- Add a `/name` command that stores the user's name and greets them by it.
- Show the total tokens used in the whole session when the user types `/quit`.
- Add a new persona called *translator* that converts everything into Hindi or Telugu.

Level 2 - Intermediate

- Save history in a JSON file and reload it when the program starts.
- Add a `/summary` command that asks the AI to summarise the chat so far.
- Add a `/resume` command that reads a text file and runs the resume-analysis template from Part 4.

Level 3 - Stretch

- Stream the reply word by word so it appears while it is being generated.
- Replace the sliding window by automatic summarising of old messages.
- Build a Tkinter or Streamlit window for the same assistant.

5.6 Quick Revision Questions

#	Question	Answer
1	What is the basic skill of an LLM?	Predicting the next token
2	Roughly how many characters is one token?	About 4 characters
3	Which role sets the AI's personality?	system
4	What does a low temperature do?	Makes output focused and repeatable
5	Why must we resend history?	The API is stateless - it has no memory
6	Where should an API key be stored?	In a .env file / environment variable
7	What is few-shot prompting?	Giving solved examples inside the prompt
8	Which error means 'too many requests'?	RateLimitError (status 429)
9	What is exponential backoff?	Waiting longer after each failed retry
10	Why delimit pasted text in a prompt?	To separate data from instructions

5.7 Glossary

Term	Meaning
Generative AI	AI that creates new content such as text, images or code
LLM	Large Language Model - a neural network trained to predict text
Token	Small unit of text (about 4 characters) that a model reads and writes
Context window	Maximum number of tokens the model can handle in one request
Prompt	The input text and instructions given to the model
System prompt	Standing instructions that set the AI's role and rules
Temperature	Setting that controls how random the output is
Hallucination	Confident but incorrect or invented AI output
API / API key	A doorway between programs / the secret that identifies your account
Environment variable	A value stored outside the code, read with os.getenv()
Stateless	Does not remember previous requests

5.8 Day 4 Summary

- **Generative AI** creates new content; **LLMs** do it by predicting the next **token**.
- The **context window** limits how much text the model sees; the API has no memory, so you resend the history.
- **Prompt engineering** = role + task + context + rules + format. Use few-shot examples, delimiters, context injection and templates.
- Call AI from Python with an **API key** stored safely in **environment variables**.
- Always wrap API calls in **try / except**, retry temporary errors, and show friendly messages.
- The same building blocks power chatbots, resume analysers, content writers, study tutors and interview coaches.

Cheat sheet

▶ **The whole idea in a few lines**

```
load_dotenv()                                  # read .env
client = anthropic.Anthropic()                 # create client
history.append({"role": "user", "content": text})
response = client.messages.create(
    model=MODEL, max_tokens=500, temperature=0.7,
    system="You are ...", messages=history)
reply = response.content[0].text
history.append({"role": "assistant", "content": reply})
```

TIP: Next steps

Keep experimenting: change the personas, lower and raise the temperature, and test your prompts with many different inputs. Good AI applications come from careful prompts, careful error handling, and a lot of testing.

NOTE: Documentation

Model names, prices and limits change over time. Always check your provider's official documentation (for Claude: docs.claude.com) before building something new.